Linked List FRQ Guide (Condensed)

1. Starter Code

Use this if the prompt does not give a node type:

```
struct Node {
    int data;
    Node* next;
    Node(int d, Node* n = nullptr) : data(d), next(n) {}
};
```

Generic skeleton:

```
Node* solve(Node* head) {
    if (head == nullptr) return nullptr;
    return head;
}
```

2. Read the Prompt Fast

Write these 5 answers before coding:

Question	Why it matters
What do I return?	Node*, int, bool, or both?
Can head change?	Needed for insert/delete at front or reverse
Read only or modify?	Tells you traversal vs pointer updates
Do I need size?	Count it or update it
What edge cases exist?	Empty, 1 node, head case, tail case, even length

If the signature is missing, choose a reasonable one and state your assumption.

3. Pattern ID Map

Prompt words	Use this pattern	Key pointers
count, sum, max, search, contains	traversal	curr
remove, delete, erase	delete + reconnect	curr, prev
insert, add, sorted insert	insert + reconnect	curr, newNode
middle, center	slow/fast	slow, fast, maybe prev
kth from end	fast gap	slow, fast
reverse	pointer flip	prev, curr, next
split, partition	build 2 lists	extra heads/tails
merge sorted lists	dummy + tail	tail
final size/count	count or update	size, int& size

4. Edge Cases to Check Every Time

Always think about:

- `head == nullptr`
- `head->next == nullptr`
- deleting the first node
- inserting at the first node
- deleting the last node
- even-length list
- whether removed nodes should be `deleted`

5. Full-Credit Checklist

Before finishing, verify:

- right pattern used
- correct loop condition
- head case handled
- no skipped nodes after deletion
- `delete` used if removing nodes
- return value is correct
- `size` updated if asked

6. Core Templates

A. Traverse

```
Node* curr = head;
while (curr != nullptr) {
    curr = curr->next;
}
```

B. Count

```
int size = 0;
for (Node* curr = head; curr != nullptr; curr = curr->next) size++;
```

C. Search

```
bool contains(Node* head, int target) {
    for (Node* curr = head; curr != nullptr; curr = curr->next)
        if (curr->data == target) return true;
    return false;
}
```

D. Delete Current Node Safely

```
if (curr == head) {
    head = head->next;
    delete curr;
    curr = head;
} else {
    prev->next = curr->next;
    delete curr;
    curr = prev->next;
}
```

Rule: if you delete `curr`, do not move `prev`.

E. Remove First Match

```
Node* removeFirst(Node* head, int target) {
    Node* curr = head;
    Node* prev = nullptr;
    while (curr != nullptr) {
        if (curr->data == target) {
            if (curr == head) head = head->next;
            else prev->next = curr->next;
            delete curr;
            return head;
        }
        prev = curr;
        curr = curr->next;
    }
    return head;
}
```

F. Remove All Matches

```
Node* removeAll(Node* head, int target) {
    Node* curr = head;
    Node* prev = nullptr;
    while (curr != nullptr) {
        if (curr->data == target) {
            if (curr == head) {
                head = head->next;
                delete curr;
                curr = head;
            } else {
                prev->next = curr->next;
                delete curr;
                curr = prev->next;
            }
        } else {
            prev = curr;
            curr = curr->next;
        }
    }
    return head;
}
```

G. Insert at Front

```
Node* insertFront(Node* head, int value) {
    return new Node(value, head);
}
```

H. Insert After a Node

```
void insertAfter(Node* curr, int value) {
    if (curr == nullptr) return;
    curr->next = new Node(value, curr->next);
}
```

I. Sorted Insert

```
Node* insertSorted(Node* head, int value) {
    Node* newNode = new Node(value);
    if (head == nullptr || value <= head->data) {
        newNode->next = head;
        return newNode;
    }
    Node* curr = head;
    while (curr->next != nullptr && curr->next->data < value) curr = curr->next;
    newNode->next = curr->next;
    curr->next = newNode;
    return head;
}
```

J. Reverse

```
Node* reverse(Node* head) {
    Node* prev = nullptr;
    Node* curr = head;
    while (curr != nullptr) {
        Node* next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}
```

Order matters: save `next`, flip pointer, move `prev`, move `curr`.

K. Find Middle

```
Node* slow = head;
Node* fast = head;
while (fast != nullptr && fast->next != nullptr) {
    slow = slow->next;
    fast = fast->next->next;
}
```

This lands on the second middle in an even-length list.

L. Remove Middle

```
Node* removeMiddle(Node* head) {
    if (head == nullptr || head->next == nullptr) {
        delete head;
        return nullptr;
    }
    Node* slow = head;
    Node* fast = head;
    Node* prev = nullptr;
    while (fast != nullptr && fast->next != nullptr) {
        prev = slow;
        slow = slow->next;
        fast = fast->next->next;
    }
    prev->next = slow->next;
    delete slow;
    return head;
}
```

M. Kth From End

```
Node* kthFromEnd(Node* head, int k) {
    Node* slow = head;
    Node* fast = head;
    for (int i = 0; i < k; i++) {
        if (fast == nullptr) return nullptr;
        fast = fast->next;
    }
    while (fast != nullptr) {
        slow = slow->next;
        fast = fast->next;
    }
    return slow;
}
```

N. Merge Two Sorted Lists

```
Node* mergeSorted(Node* a, Node* b) {
    Node dummy(0);
    Node* tail = &dummy;
    while (a != nullptr && b != nullptr) {
        if (a->data <= b->data) {
            tail->next = a;
            a = a->next;
        } else {
            tail->next = b;
            b = b->next;
        }
        tail = tail->next;
    }
    tail->next = (a != nullptr) ? a : b;
    return dummy.next;
}
```

7. Which Template Do I Use?

If the task says...	Start from...
remove nodes by condition	removeAll
remove one node	removeFirst
middle	find middle or removeMiddle
reverse	reverse
kth from end	kthFromEnd
insert in order	insertSorted
compare or inspect values only	traverse
combine 2 sorted lists	mergeSorted

8. Worked FRQ: Remove Middle and Track Final Size

Reasoning:

- "middle" -> slow/fast
- "remove" -> need prev
- "final size" -> decrement `size`

Safe assumption: return `Node*`, pass size by reference.

```
Node* removeMiddle(Node* head, int& size) {
    if (head == nullptr) {
        size = 0;
        return nullptr;
    }
    if (head->next == nullptr) {
        delete head;
        size = 0;
        return nullptr;
    }
    Node* slow = head;
    Node* fast = head;
    Node* prev = nullptr;
    while (fast != nullptr && fast->next != nullptr) {
        prev = slow;
        slow = slow->next;
```

```
        fast = fast->next->next;
    }
    prev->next = slow->next;
    delete slow;
    size--;
    return head;
}
```

Dry run on `1->2->3->4->5`: remove 3, return `1->2->4->5`, size becomes 4.

9. Common Mistakes

Mistake	Why it loses points
prev->next = curr->next when curr == head	prev is null
delete curr; curr = curr->next;	uses freed memory
moving prev after deleting curr	skips nodes
reversing without saving next first	loses rest of list
forgetting to return new head	wrong answer after front insert/delete/reverse

10. Panic Templates

Delete-by-condition template

```
Node* solve(Node* head) {
    Node* curr = head;
    Node* prev = nullptr;
    while (curr != nullptr) {
        if (/* delete condition */) {
            if (curr == head) {
                head = head->next;
                delete curr;
                curr = head;
            } else {
                prev->next = curr->next;
                delete curr;
                curr = prev->next;
            }
        } else {
            prev = curr;
            curr = curr->next;
        }
    }
    return head;
}
```

Reverse template

```
Node* solve(Node* head) {
    Node* prev = nullptr;
    Node* curr = head;
    while (curr != nullptr) {
        Node* next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}
```

11. Final 20-Second Check

1. Did I pick the right pattern?
2. Does the empty-list case work?
3. Does the head case work?
4. Are pointer updates in the right order?
5. Did I return the correct thing?